

CUDA PROGRAMMING FOR NVIDIA H100S

异步与屏障

Prateek Shukla · 简体中文版

同步世界中的系统

在任何系统中都有两种主要操作

做（Doing）：处理信息、解方程、做饭

取（Fetching）：取回数据、阅读、理解问题、拿食材

在同步世界里，这些操作顺序发生。在当前任务完成之前，你会被"阻塞"。

用同步方式做饭是指：你在做饭的同时不对任何其他工作做多任务处理。饭做好了，你才去做下一个任务。你被"阻塞"，无法做任何其他事情。

异步将请求与结果解耦。

做饭时，你出去做别的事情，过一段时间，在某种信号到来后回来使用做好的饭。

你用做完一件事的时间完成了两件事。你就实现了延迟隐藏。

如果获取一项资源的成本高于处理它所需的时间，你就必须把下一项的获取与当前项的处理重叠起来。

延迟隐藏的本质是不被一条指令"阻塞"，并且有能力把当前工作放到后台，去做别的事情。

阻塞与非阻塞操作

阻塞或同步操作会暂停发起线程（例如 CPU host 线程）的执行，直到该操作完全完成。控制权不会返回给线程，因此它无法继续执行下一行代码。这形成了一个隐式同步点，保证在 host 继续之前任务已经完成。

非阻塞或异步操作会在操作真正完成之前，立即把控制权交还给发起线程。

GPU 中的异步性

在现代计算中，“做”（数学/计算）快得难以置信。相比之下，“取”（内存访问）慢得令人痛苦。

取数据所花的时间远高于对数据做计算所花的时间。因此高性能体系结构的目标不只是让数学运算更快；而是确保数学运算永不停止。

Nvidia H100 常因其原始速度（FLOPS）而受到称赞，但它真正的天才之处在于其异步性体系结构。它的设计确保其庞大的张量核心永远不必等待数据。这靠的是对张量核心与 TMA 使用非阻塞指令。

异步操作遵循这个模式

Stage 1 : 初始化 - 一个线程发起一个异步操作，然后立即转向下一条指令

Stage 2 : 跟踪与并行执行 - 系统的某一部分跟踪该操作（TMA 用 mbarrier，wgmma 用内部硬件记分板）。这些操作运行期间，其他单元并行工作

Stage 3 : 同步 - 一旦异步操作在后台完成运行

H100 中的同步步骤

warp 发起指令的速度非常快，在几纳秒量级

一旦异步指令到达执行单元，它会立即尝试读取内存

瓶颈是内存带宽。把数据从 HBM 搬到共享内存/寄存器是一个高延迟操作，与逻辑核心相比需要几百纳秒。

两个问题

由于发起者 (Issuer, 约 ns 级) 比内存搬运者 (Memory Mover, 约几百 ns 级) 更快, 如果拷贝指令与计算指令同时发出, 张量核心就必须为数据的到来等待几百纳秒。因此存在延迟隐藏的空间。

此外, 在一个大 kernel 中, 指令队列里塞满了针对尚未到达数据的命令, 如果没有某种形式的同步, 张量核心可能会在数据还没到达时就执行操作。

我们需要一种办法确保正确的操作作用在正确的数据上, 同时我们也需要一个可以施展延迟隐藏的系统。

mbarrier/信号量究竟是什么

一种驻留在共享内存中的硬件加速同步原语，设计用来跟踪异步内存事务的完成。

传统屏障阻塞执行直到线程到达，而 `mbarrier` 阻塞执行直到数据到达。它把"生产者"（发起拷贝的一方）与"消费者"（等待数据的一方）解耦，从而实现分相（split-phase）、发射后不管（fire-and-forget）的内存流水线。

生产者在屏障中设定预期的数据传输量，然后硬件在后台完成它的工作；事务一旦完成，屏障就被打开。

为什么这是一个很好的解决方案

屏障迫使"快"的指令发起者尊重"慢"的物理硬件，方式是：

1. 确保较慢的操作独立于较快的操作发起，这样等到快单元被启动时，数据已经就位。
2. 有办法防止快的执行单元在错误的数据上操作。

而我们做到这一点的方式就是使用 `mbarrier`。

CUDA PROGRAMMING FOR NVIDIA H100S

总体图景



Prateek Shukla · 简体中文版

CUDA 中的 **proxy**

在 NVIDIA H100 与 CUDA PTX 内存模型的语境下，Proxy 是一个用来区分谁在执行内存操作的术语。

如果两个内存操作发生在同一个 proxy 中（例如 Generic Proxy），硬件会保证它们是安全且有序的（大体上）。

如果操作 A 在 Proxy 1、操作 B 在 Proxy 2，硬件就不再检查。它假设两者完全无关，任由它们不受约束地乱序、并行运行。

Generic proxy 与 Async proxy

当你编写 CUDA kernel 时，你的线程按顺序执行指令。当线程读或写内存时，它扮演的是 Generic Proxy。硬件保证这些操作在该特定线程内按你书写它们的顺序发生（大体上）。

Async Proxy 指的是执行批量异步拷贝或张量核心数学运算的硬件机制。当你发出 `cp.async.bulk` 或 `wgmma` 这样的命令时，Generic Proxy（线程）只是启动该命令，然后立即转向下一行代码。

Async Proxy 完全独立于 Generic Proxy 运行。它不知道线程当前在做什么，线程也不会自动知道 proxy 何时完成。

写后读 (**Read After Write**)

Generic Proxy 与 Async Proxy 通往内存的路径不同。Generic Proxy 通过 SM 的 L1 缓存工作，而 Async Proxy 绕过 L1，直接与 L2 缓存或 HBM 交互。

Generic Proxy 发出的存储最初停留在本地 store 缓冲或 L1 缓存中。它们不会立即对系统其余部分（包括 TMA）可见。

如果 Generic Proxy 写入某个内存地址，并立即触发 Async Proxy 去读同一地址，Async Proxy 很可能会从 L2/DRAM 读到过期数据，因为新数据还卡在 Generic Proxy 的 L1 里。

这称为写后读 (read after write) 冒险。

读后写 (**Write After Read**)

一个线程 (或 CTA) 通过 Generic Proxy 读取地址 X , 把 X 拉入 L1 (或者以其他方式在本地"锚定"一个旧版本) 。

之后 , 一次 Async Proxy 写入通过 L2/HBM 更新了 X , 却没有更新/失效 Generic Proxy 的 L1 视图。Generic Proxy 可以继续到过期的 L1 行上操作。两种常见失败模式 :

之后 Generic Proxy 的存储/写回/逐出可能会覆盖 Async Proxy 写入的较新值 (过期行获胜) , 或者 Generic Proxy 后续的加载不断返回旧值 , 尽管 X 已经通过 Async Proxy 更新。

这称为读后写 (write after read) 冒险 , 即 WAR。

栅栏 (**fences**)

栅栏是一个显式的排序/可见性点，约束内存效果如何变为可观察。当硬件可能让事情彼此"越过"时，你就要使用它——尤其是那些不会像你假设的那样自动与普通加载/存储排序的异步操作。

NVIDIA 明确指出：要跨 proxy 同步以获得正确的排序，需要 proxy 栅栏。

此外这些栅栏是带作用域的（例如 .cta、.cluster、.gpu、.sys），作用域决定谁必须看到这个排序，并与层级中的某个一致性点（例如 L1 与 L2）绑定。

栅栏有两类：普通栅栏与跨 proxy 栅栏。

Release 栅栏（生产者侧）

Release 栅栏强制执行这条单向规则：

在程序顺序中出现在 release 之前的所有内存操作（尤其是写入），对于与它同步（在选定作用域内）的其他线程来说，必须在出现在 release 之后的任何操作之前变为可见。

它防止较早的写入被延迟/重排到 release 点之后。

Acquire 栅栏（消费者侧）

Acquire 栅栏强制执行相反的单向规则：

在程序顺序中出现在 acquire 之后的内存操作（尤其是读取），不允许被观察到发生在它之前。

在 acquire 之后，保证线程能观察到由匹配的 release 变为可见的那些写入（同样，在选定的作用域/proxy 内）。

跨 **proxy** 栅栏

如果你跨多个 proxy 访问同一位置，就需要跨 proxy 栅栏。对于 async proxy，使用 `fence.proxy.async` 在 generic proxy 与 async proxy 之间同步内存。

它排空/排序 generic proxy 的共享内存写入可见性，使 async proxy 不会读到较旧的视图。

它不是"集体刷新"。它是每线程的排序，所以你仍然需要一次线程块同步，确保所有写入者都已执行完毕，然后由当选线程发起 TMA（CUDA 示例在准备/协调代码周围使用 `__syncthreads()`）。

完整的 **mbarrier** 流水线

第 1 步：在共享内存中初始化 mbarrier，使用 mbarrier.init 创建一个带有预期线程到达数的屏障。线程到达数确保特定数量的线程已经发出了拷贝指令。第 2 步：使用 mbarrier.arrive 为你将要发起的异步操作设定预期事务数。每次这样做，你都在登记一条指令，从而减少到达数。第 3 步：发起异步操作第 4 步：发起其他操作并等待（线程休眠，直到线程到达满足且 `expected_tx = 0`）第 5 步：翻转相位，重置到达数，进行下一个操作

相位与预期事务数

屏障的相位指的是它当前可复用的状态或周期。它是一个单独的 bit，每当屏障完成一个周期就翻转一次。事务数表示你正在执行的异步操作的规模。预期事务数是异步操作尚待完成的"工作"量。由于这些是异步操作，硬件会随着异步操作的进展自动递减事务数。我们把屏障附加到异步操作上来确保这一行为。你可以在操作结束时翻转相位并增加事务数，从而"复用"屏障。线程到达数会按照初始化时设定的值自动重置。

初始化 **mbarrier**

这是大多数 bug 唯一的起源指令。这一步搞错，其他一切都不再重要。

addr 就是 mbarrier 在状态空间（state space）中的内存地址。

Count 是屏障的预期线程到达数；当你通过翻转相位复用屏障时，屏障就会重置为这个值。

需要记住的几点

通常作用域是 `shared::cta`，这样只有同一线程块中的线程才能"看到"屏障。

地址是共享内存指针，用 `__cvta` 创建。

你可以使用 `shared::cluster` 让屏障对集群中的所有线程可见。

你必须使用 `mapa` 这条 PTX 指令来获取集群中的地址。

`mbarrier` 对象必须 64 位对齐，未对齐访问可能导致静默损坏。

设定预期事务数

把到达数减 1。

用 `tx_count` 增加预期事务字节数。

`tx_count` 会被加到屏障的未决 tx-count 上。如果你用 4096 调用两次 `arrive.expect_tx`，屏障总共预期 8192 字节。

占位符 `_`

指令中的 `_` 是 `phase_out` 的占位符；`phase_out` 是获取一个 64 位编码 token 的方式，该 token 包含屏障当前相位与事务数的信息。

它存在于一些复杂的同步问题中。

我们丢弃 `phase_out`，因为我们实际上是把这些信息写进生产者线程。寄存器压力是昂贵的。翻转一个 1 位整数比在寄存器里保存一个 64 位 token 更便宜。

同步

到目前为止，我们已经发起了异步拷贝指令，但现在我们怎么知道操作已经完成了。

老办法是 `barrier.sync`，它会阻塞线程直到操作完成，但这会摧毁异步性。

在 Hopper 上我们的做法是：有一条指令能够给出操作是否完成的真或假。这使延迟隐藏成为可能。线程可以检查屏障，发现还没就绪就重复检查，直到操作结束、相位翻转。这正是 `mbarrier.try_wait.parity` 的工作。

mbarrier.try_wait.parity

你向指令传入一个 phaseParity 位（0 或 1），表示在继续之前你必须验证已完全完成的那个特定执行相位。

硬件将你输入的奇偶性位与 mbarrier 对象当前的内部奇偶性状态进行比较，以判断那个特定相位是否仍在进行中。

如果你输入的奇偶性等于屏障当前的奇偶性，屏障仍在处理那个相位，指令返回 false（继续等待）。如果你输入的奇偶性与屏障当前的奇偶性不同，屏障已推进到下一个相位，指令返回 true（继续执行）。

成功返回意味着屏障奇偶性已经"翻转"，也就是说你请求的奇偶性现在指的是紧邻的前一个（因此已完成的）相位。

这条指令

`try_wait` : 操作名。它暗示一次可能涉及线程暂时挂起的尝试。

`waitComplete` : (目标) 布尔结果。

1 (真) : 屏障已到达目标相位 (工作已完成)。

0 (假) : 屏障尚未完成 , 线程刚刚被唤醒。

`suspendTimeHint` : 可选的立即数 (常量) , 告诉 GPU 调度器在条件不满足时让出线程多长时间。

`phaseParity` : 一个整数 (0 或 1) , 表示你正在等待的相位世代。

mbarrier.test_wait.parity

`test_wait.parity`：该指令的变体。它使用"相位奇偶性"位（0 或 1）而不是原始整数计数来跟踪屏障的进度。

`waitComplete`：（目标）一个 1 位谓词寄存器（布尔值）。屏障完成时收到 1，仍在忙时收到 0。

`[addr]`：（源）指向共享内存中 `mbarrier` 对象的指针/地址。

`phaseParity`：（源）一个整数（0 或 1），表示你正在等待的相位世代。

带 `.acquire` 的 `mbarrier` 等待

它做了 `mbarrier.try_wait` 所做的一切，但有一个关键区别。

如果当前同步轮次已完全结束，`.acquire` 语义会创建一个严格的内存栅栏，确保来自 `async proxy` 或其他线程的所有数据写入在继续之前保证可见。

如果你正在把数据从共享内存加载到寄存器（以手动喂给张量核心），`.acquire` 确保那些 LD 指令在数据有效之前不会发射。如果它们过早发射，你的寄存器里就是垃圾。

即使 `wgmma` 直接从共享内存读取，这条指令本身也要求描述符与内存状态保持一致。`.acquire` 确保依赖链得到尊重。

CUDA PROGRAMMING FOR NVIDIA H100S

实现它的复杂方式

Prateek Shukla · 简体中文版

一个重要的点

在发起拷贝的生产者 warp 中，只有一个线程在运行所有指令，比如 `mbarrier.init`、`mbarrier.arrive.expect_tx`、`cp.async.bulk` 等等。warp 里的其他线程只是闲着。

消费者 warp 才是发起等待操作的一方，因为它们在等 tma 拷贝完成，以便处理这些数据。

mbarrier 复用模式

相位声明在寄存器中，而不是共享内存里！每个线程维护自己的 `int phase`，并以 `phase ^= 1` 作为循环中的最后一个操作。

mbarrier 只适用于异步批量拷贝吗？？

到目前为止我们讨论了 `mbarrier` 与 `cp.async.bulk`，确实让人觉得 `mbarrier` 就是为异步批量拷贝而生的。考虑这个情形：生产者写入共享内存消费者从共享内存读取（使用 `WGMMA`）。生产者想用下一个 `tile` 覆写同一块共享内存。如果生产者在 `WGMMA` 还在读 `sA` 时就覆写它，你的数学结果就是垃圾。这是一个读后写（`Write-After-Read`，`WAR`）冒险。

我们如何防止 **WAR**

我们创建一个带有预期线程到达数的屏障。

消费者忙着读数据，生产者在屏障上自旋。

消费者完成它最后一条读指令。

消费者执行减少线程到达数的指令。

屏障状态翻转，现在生产者可以写入新数据。

减少到达数的指令是.....

mbarrier.arrive.scope

只是把屏障未决的到达线程数减少 count。

编译器与重排

编译器常常会重排操作，以获得一些效率提升。

```
a = 1;
```

```
b = 2;
```

```
c = a + b;
```

编译器可能先做 $b = 2$ 再做 $a = 1$ ，因为它们认为这不影响使用这两个值的整个 $c = a + b$ 操作。

为什么这在异步 **GPU** 操作中是致命的

在多线程代码中，重排会破坏正确性，因为不同线程可以以不同的顺序观察到这些操作。线程 a - `d[0][0] = 42.0f; flag = 1;` 线程 b - `while (flag == 0); float x = d[0][0];`

mbarrier.arrive.release.scope

.release 后缀提供 release 语义。这意味着它充当一个单向硬件栅栏：这一行之上的内存写入不会重排到它之下。

在线程完成生成其他线程将要消费的数据之后，使用 `mbarrier.arrive.release`：

销毁屏障

它正式将一个 mbarrier 对象（一种 64 位同步原语）置为无效，实质上清除了硬件对该屏障状态的跟踪。

通过使屏障失效，它释放了那个特定的共享内存地址，使其可以在 kernel 的后续部分被安全覆写或改作其他变量之用。

它还专门把一个 generic 指针转换为 32 位共享内存偏移，以确保 GPU 硬件寻址到正确的本地内存 bank。

完整流水线

1. `mbarrier.init` : 线程在共享内存中创建屏障，设定预期的线程数，并从 Phase 0 开始
2. `mbarrier.expect_tx` : 生产者线程告诉屏障，还要预期即将到来的异步操作带来的特定字节数
3. `cp.async.bulk` : 一个线程发起 `cp.async` (拷贝)，然后 `mbarrier.try_wait.parity 0`，自旋直到 Phase 0 完成
4. 翻转 : 当所有预期字节 (和线程) 都到达时，屏障自动翻转到 phase 1
5. 计算开始
6. 计算结束后，我们手动翻转相位，然后开始处理下一个 k tile 以复用屏障

mbarrier.arrive_drop

它的行为类似标准的 `arrive` 操作，递减当前相位的未决到达数。

但更重要的是，它还会永久递减屏障的预期到达数。所以如果你翻转相位并试图复用屏障，“预期到达数”将是之前的值减去已发生的 `drop` 次数。

.sem .expect_tx 与 .noComplete

arrive_drop.expect_tx：设定预期事务数，并永久且临时地把到达数减 1。

arrive_drop.sem - sem 可以是 .release 或 .relaxed。 .relaxed 表示不强制任何内存排序； .release 确保线程在到达之前执行的所有内存写入，对任何在该屏障上等待的线程可见。

.noComplete：指示硬件执行到达（递减未决/预期计数）而不触发相位完成，即使完成条件（未决计数到达零）已经满足。

barrier.cluster

像 `bar.sync (__syncthreads)` 这样的标准屏障只能同步单个线程块内的线程。它们无法协调跨集群的生产者/消费者线程块。

要同步在同一集群上共同调度的不同线程块，需要 `barrier.cluster`。它还保证屏障之前对 `DSMEM` 的任何写入，在屏障之后对集群中的所有块可见。

关键用例是 TMA 多播。它的工作方式是：在最初的阶段，线程块 0 可能正在运行而线程块 1 可能尚未运行，如果线程块 0 试图写入线程块 1，程序就会崩溃。通过发起这个屏障，你确保每个块都已实际就位。

barrier.cluster 中两条重要的指令

`barrier.cluster.arrive`：线程发出信号表示它已到达屏障。它并不停下，而是继续执行不依赖其他块数据的独立指令（数学运算、本地寄存器操作）。

`barrier.cluster.wait`：这是一条阻塞指令。线程在此停住，直到集群中所有其他线程/块都发出到达信号。一旦这里解除阻塞，你就能保证其他块写入的所有数据现在可以安全读取。

在 H100 上，由于块 A 可以写块 B 的内存，我们需要一个屏障来防止块 B 在块 A 写完之前读取。

异步组

当你使用 `bulk group` 发起一个 `cp.async.bulk` 操作时，它看起来像这样 -

`Bulk_group` 附加在 `cp.async.bulk` 指令上，这使我们能够使用 `cp.async.bulk.commit_group` 与 `cp.async.bulk.wait_group`。

这些就是我们将用来把一批指令打包成批、然后再使用它们的指令。

cp.async.bulk.commit_group

这里流水线的工作方式是：我们使用带 `bulk_group` 的 `cp.async.bulk` 发起一批操作。

由于之前没有执行过 `cp.async.bulk.commit_group` 操作，这些拷贝是未提交的；我们可以提交，即把它们打包进一个组。把它们打包成组的要点在于：之后我们可以让其他单元等待，直到 N 个单元执行完毕。

在这条指令之后发起的任何指令都属于下一个组，或者就只是普通指令。

我们可以拥有多个组，每组包含多个 `cp.async.bulk` 操作。

cp.async.bulk.wait_group<N>

一旦我们发起并提交了一批操作，就需要让将要使用它们输出的其他单元来等待。

`wait_group<N>` 指令等待直到最多只剩 `N` 个已提交的组处于未决状态。例如，`wait_group<0>` 等待所有组完成，而 `wait_group<2>` 表示等待到你发起的所有操作中只剩两个仍处于未决状态。

这个计数指的是仍未决的组，而不是已完成的组。所以 `wait_group<2>` 的意思是"等待直到只有最近的 2 个已提交组仍未决"，所有更早的组必须已经完成。

cp.async.bulk.wait_group.read

它既停顿执行，又强制执行一个 acquire 栅栏。

这对写后读（read-after-write）场景非常重要，否则你有可能读到旧数据。

命名屏障 (**namedbarriers**)

`__syncthreads()` 是一次全块会合。命名屏障让你可以在一个块内创建多个独立的同步点，让不同的 warp 子集相互协调，而不强迫无关的 warp 停下。

PTX 明确允许不同的 warp 对同一个命名屏障使用不同的操作，例如混合使用 `.arrive` 与 `.sync` 来构建生产者/消费者流水线。

`a` = 屏障 ID , `b` = 参与线程数